

8. LIST AND ITERATOR ADTS

List concepts

List applications

A list ADT: requirements, contract

Iterators

Implementations of lists: using arrays and linked-lists

Lists in the Java class library

List concepts (1)

- A **list** is a sequence of elements, in a fixed order. Elements can added/removed at any position.
- Elements are in positions 0 (leftmost), 1, 2,
- The **size** (or **length**) of a list is the number of elements.
- The **concatenation** of lists l_1 and l_2 is a list containing all elements of l_1 followed by all elements of l_2 .
- We can **traverse a list (or iterate over the list)**, i.e., visit each of the list's elements in turn.

List concepts (2)

- Mathematical notation for lists:
 - $\langle x_0, x_1, \dots, x_{n-1} \rangle$ is a list of length n , whose elements are $x_0, x_1, \dots, x_{n-1}$ (in that order).
 - $\langle \rangle$ is the empty list.
 - *Note:* We can use this notation in algorithms, but it is **not** supported by Java.

Example: lists and concatenation

- A list of integers:

fibonacci = «1, 1, 2, 3, 5, 8, 13, 21, 34»

- A list of airports:

tour = «GLA, LHR, CDG, GLA»

- Lists of words:

hamlet1 = «'to', 'be', 'or', 'not', 'to', 'be'»

hamlet2 = «'that', 'is', 'the', 'question'»

- Concatenation of those lists of words:

«'to', 'be', 'or', 'not', 'to', 'be', 'that', 'is', 'the', 'question'»

Lists vs linked-lists

- Do not confuse the **list** *abstract data type* with **linked-list data structures**.
- A list ADT can be implemented using different data structures (arrays, linked-lists).
- Conversely, linked-list data structures can be used to implement many different ADTs (e.g., stacks, queues, lists, sets).

List applications

- A sentence is a list of words.
 - The words are in the order they are read or spoken.
- An itinerary is a list of places visited on a tour.
 - The places are in the order they are visited.
- A log is a list of event records (e.g., equipment faults).
 - The event records are in time order.

Example: simple text editor (1)

- Consider a very simple text editor that supports insertion and deletion of complete lines only.
- The user can load text from a file, or save the text to a file.
- The user can select any line of the text:
 - directly (by pointing at it, or giving a line number)
 - by searching for a line matching a given search string.
- The user can delete the selected line.
- The user can insert a new line, either above the selected line or below the selected line.

Example: simple text editor (2)

- We can represent the text being edited by:
 - a list of lines, *text*
 - the position *sel* of the selected line
- We can implement the user commands straightforwardly in terms of list operations, e.g.:
 - **Delete**: remove the line at position *sel* in *text*.
 - **Insert above**: add the new line at position *sel* in *text*, then increment *sel*.
 - **Insert below**: increment *sel*, then add the new line at position *sel* in *text*.
 - **Save**: traverse *text*, writing each line to the output file.

List ADT: requirements

- Requirements:
 - 1) It must be possible to make a list empty.
 - 2) It must be possible to test whether a list is empty.
 - 3) It must be possible to obtain the length of a list.
 - 4) It must be possible to add an element at any position in a list.
 - 5) It must be possible to remove the element at any position in a list.
 - 6) It must be possible to inspect or update the element at any position in a list.
 - 7) It must be possible to concatenate lists.
 - 8) It must be possible to test lists for equality.
 - 9) It must be possible to traverse a list.

List ADT: contract (1)

- Possible contract for homogeneous lists:

```
public interface List<E> {  
    // Each List<E> object is a homogeneous list  
    // whose elements are of type E.  
    ////////////////////////////////// Accessors //////////////////////////////////  
    public boolean isEmpty ();  
    // Return true if and only if this list is empty.  
    public int size ();  
    // Return this list's length.
```

List ADT: contract (2)

- Possible contract (*continued*):

```
public E get (int p);  
// Return the element at position p in this list.  
public boolean equals (List<E> that);  
// Return true if and only if this list and that have the  
// same length, and each element of this list equals  
// the corresponding element of that.
```

List ADT: contract (3)

- Possible contract (*continued*):

////////// Transformers //////////

```
public void clear ();
```

```
// Make this list empty.
```

```
public void set (int p, E it);
```

```
// Replace the element at position p in
```

```
// this list by it.
```

```
public void add (int p, E it);
```

```
// Add it at position p in this list.
```

```
public void addLast (E it);
```

```
// Add it after the last element of this list.
```

----- This changes
the positions
of succeeding
elements.

List ADT: contract (4)

- Possible contract (*continued*):

```
public void addAll (List<E> that);
// Add all the elements of that after the
// last element of this list.
```

```
public E remove (int p);
// Remove and return the element at
// position p in this list.
```

```
////////// Iterator //////////
```

```
public Iterator<E> iterator ();
// Return an iterator that will visit all
// elements of this list, in left-to-right order.
```

```
}
```

----- This changes
the positions
of succeeding
elements.

Traversal (1)

- To traverse `array`:

```
for (int i = 0; i < array.length; i++)  
    ... array[i] ...
```

This traversal has time complexity $O(n)$.

- We could mimic this to traverse `list`:

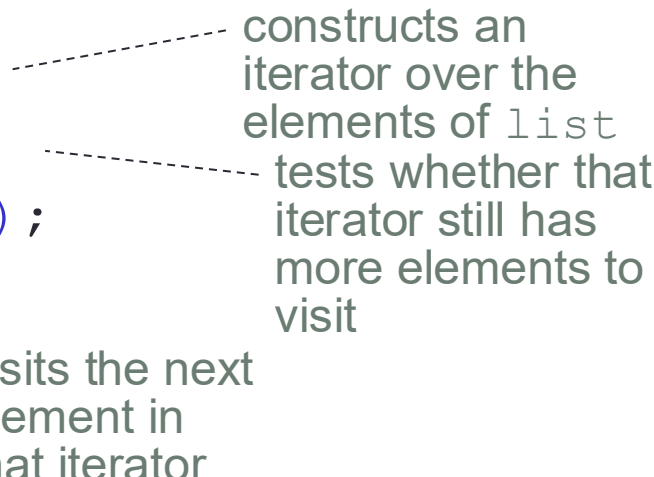
```
for (int p = 0; p < list.size(); p++)  
    ... list.get(p) ...  
    ... list.set(p, x) ...
```

- But this traversal could have time complexity $O(n^2)$, if `get` and `set` turn out to be $O(n)$.

Traversal (2)

- Better, use an **iterator** to traverse `list`:

```
Iterator<T> elements =  
    list.iterator();  
while (elements.hasNext()) {  
    T elem = elements.next();  
    ... elem ...  
}
```



constructs an iterator over the elements of `list`

tests whether that iterator still has more elements to visit

visits the next element in that iterator

- This traversal has time complexity $O(n)$, since the `hasNext()` and `next()` operations are guaranteed to be $O(1)$.

Iterators (1)

- View an iterator as a path along which we visit the elements one by one, in some desired order.
- Examples of iterators over a list:

« 'to', 'be', 'or', 'not', 'to', 'be' »

----- left-to-right
iterator

----- right-to-left
iterator

Iterators (2)

- The `List` interface's `iterator()` operation constructs a left-to-right iterator over the list elements.
- The iterator's `hasNext()` operation tests whether there is a next element still to be visited.
- The iterator's `next()` operation returns the next element (if any).

Iterator ADT: contract

- Java's contract for iterators:

```

public interface Iterator<E> {
    // Each Iterator<E> object represents an iterator
    // over some collection of elements of type E.

    public boolean hasNext ();
    // Return true if and only if this iterator has a next
    // element. Guaranteed O(1).

    public E next ();
    // Return the next element in this iterator.
    // Guaranteed O(1).

    ... ----- omitted operation
}

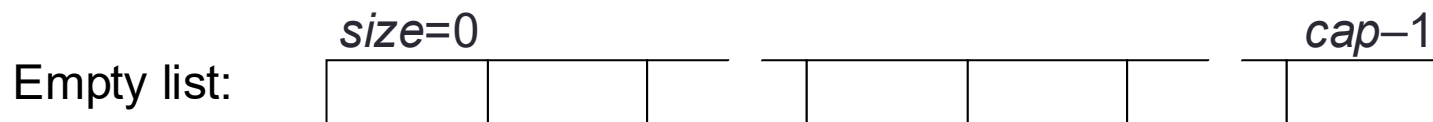
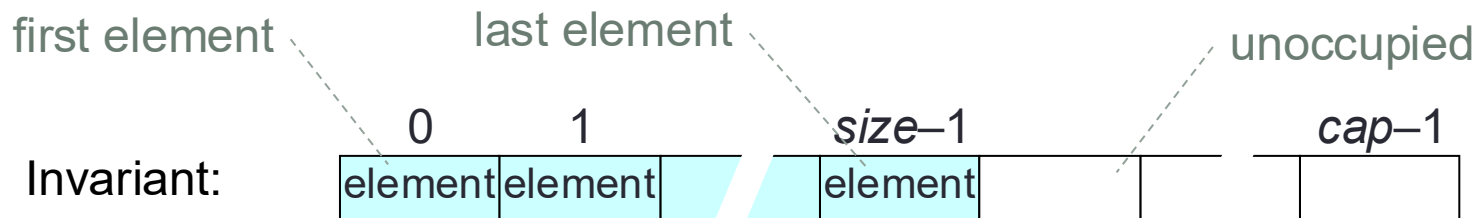
```

Iterators: implementation

- An iterator is represented by a position on the iterator's path, typically:
 - an index (if the elements are held in an array)
 - a link (if the elements are held in a linked-list).
- The `hasNext()` operation tests whether there is a next position on the iterator's path.
- The `next()` operation advances to the next position on the iterator's path, and returns the element at that position.
 - It throws an exception if there is no next position.

Implementation of lists using arrays (1)

- Represent a *bounded* list ($\text{size} \leq \text{cap}$) by:
 - a variable *size*
 - an array *elems* of length *cap*, containing the elements in *elems*[0...*size*−1].



Implementation of lists using arrays (2)

- Java implementation:

```
public class ArrayList<E>
    implements List<E> {
    private E[] elems;
    private int size;
    ////////////////////////////////// Constructor //////////////////////////////////
    public ArrayList (int cap) {
        elems = (E[]) new Object[cap];
        size = 0;
    }
```

Implementation of lists using arrays (3)

- Java implementation (*continued*):

```
////////// Accessors //////////  
public int size () {  
    return size;  
}  
  
public E get (int p) {  
    if (p < 0 || p >= size) throw ...;  
    return elems[p];  
}  
  
...
```

Implementation of lists using arrays (4)

- Java implementation (*continued*):

////////// Transformers //////////

```
public void add (int p, E it) {  
    if (p < 0 || p > size) throw ...;  
    if (size == elems.length) ...  
    for (int j = size; j > p; j--)  
        elems[j] = elems[j-1];  
    elems[p] = it;  
    size++;  
}  
...
```

Implementation of lists using arrays (5)

- Java implementation (*continued*):

```
////////// Iterator //////////
```

```
public Iterator<E> iterator () {  
    return new LRIterator();  
}
```

```
////////// Inner class //////////
```

```
private class LRIterator  
    implements Iterator<E> {  
    ...  
}  
}
```

Implementation of lists using arrays (6)

- Implementing iterators over ArrayList objects:

```
////////// Inner class //////////  
private class LRIterator  
    implements Iterator<E> {  
    // An LRIterator object is a left-to-right iterator  
    // over an ArrayList<E> object.  
  
    private int position;  
    // position is the index of the slot containing the  
    // next element to be visited.  
  
    private LRIterator () {  
        position = 0;  
    }  
}
```

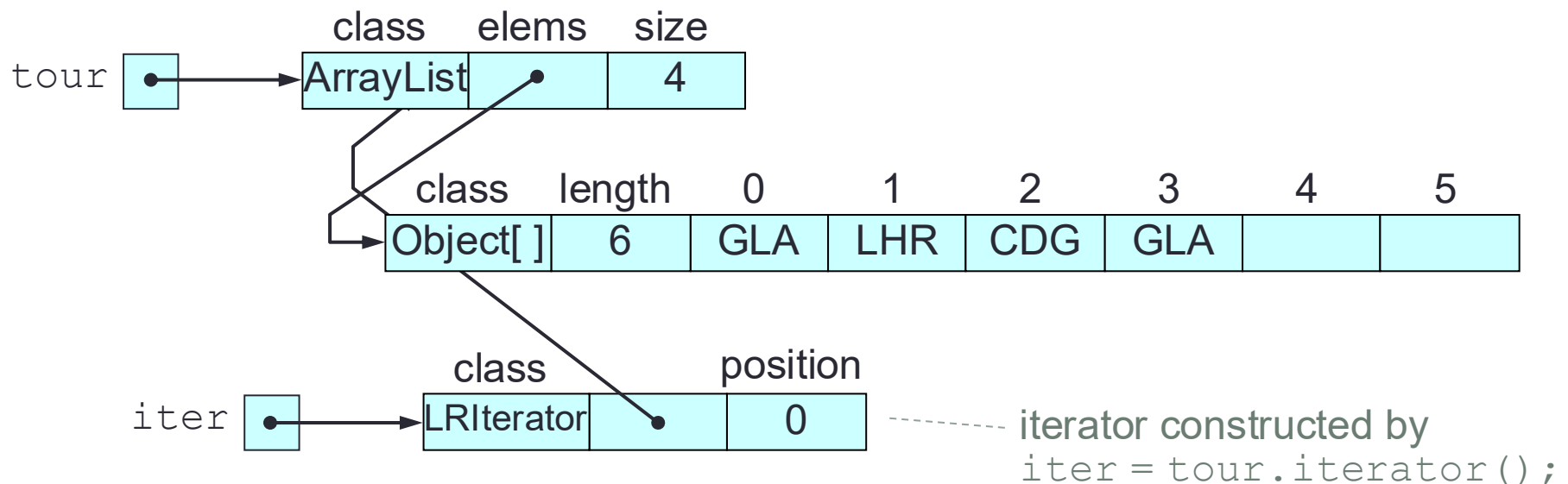
Implementation of lists using arrays (7)

- Implementing iterators over `ArrayList` objects
(*continued*):

```
    public boolean hasNext () {  
        return (position < size);  
    }  
  
    public E next () {  
        if (position >= size) throw ...;  
        return elems[position++];  
    }  
  
    ...  
}
```

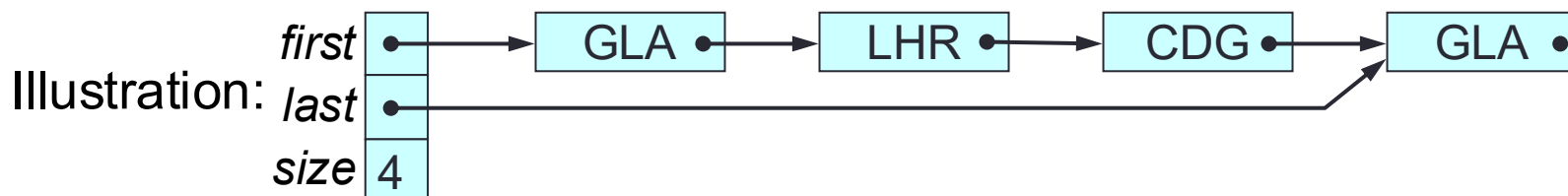
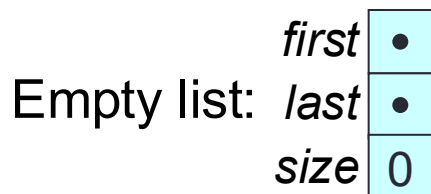
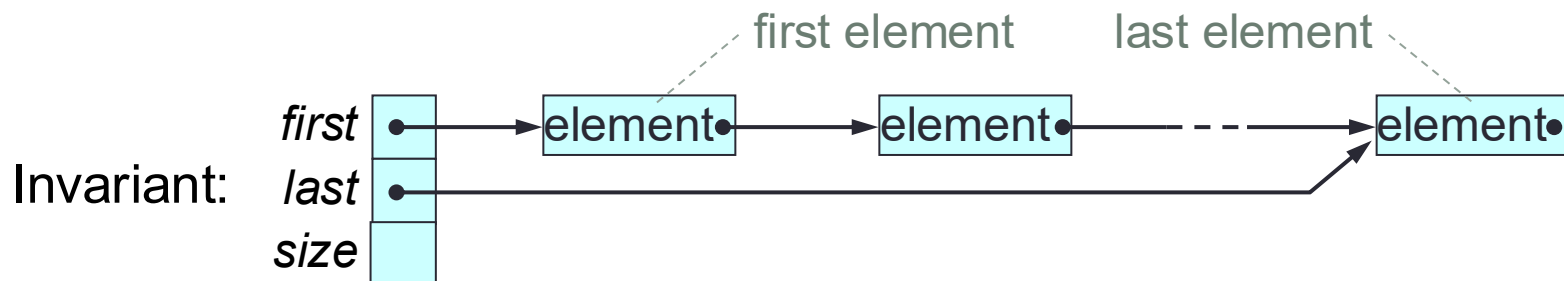
Implementation of lists using arrays (8)

- Since `LRIterator` is a non-static inner class of `ArrayList`, its methods can access `ArrayList` instance variables. E.g.:



Implementation of lists using SLLs (1)

- Represent an (unbounded) list by:
 - a variable *size*
 - an SLL, with links to both first and last nodes.



Implementation of lists using SLLs (2)

- Java implementation:

```
public class LinkedList<E>
    implements List<E> {
    private Node<E> first, last;
    private int size;
    /////////////// Inner class ///////////////
    private static class Node<E> {
        ...
    }
    /////////////// Constructor ///////////////
    public LinkedList () {
        first = last = null;
        size = 0;
    }
```

Implementation of lists using SLLs (3)

- Java implementation (*continued*):

```
////////// Accessors //////////  
public int size () {  
    return size;  
}  
  
public E get (int p) {  
    if (p < 0 || p >= size) throw ...;  
    return locate(p).element;  
}  
  
...
```

Implementation of lists using SLLs (4)

- Java implementation (*continued*):

```
////////// Auxiliary method //////////  
private Node<E> locate (int p) {  
    // Return a link to the node at position p in this list.  
    Node<E> curr = first;  
    for (int j = 0; j < p; j++)  
        curr = curr.next;  
    return curr;  
}
```

Implementation of lists using SLLs (5)

- Java implementation (*continued*):

```

//////////////////// Transformers //////////////////////
public void add (int p, E it) {
    if (p < 0 || p > size) throw ...;
    Node<E> newest = new Node<E>(it, null);
    if (p == 0) {
        newest.next = first; first = newest;
    } else {
        Node<E> pred = locate(p-1);
        newest.next = pred.next;
        pred.next = newest;
    }
    if (newest.next == null)
        last = newest;
    size++;
}

```

Implementation of lists using SLLs (6)

- Java implementation (*continued*):

```
////////// Iterator //////////
```

```
public Iterator<E> iterator () {  
    return new LRIterator();  
}
```

```
////////// Inner class //////////
```

```
private class LRIterator  
    implements Iterator<E> {  
    ...  
}  
}
```

Implementation of lists using SLLs (7)

- Implementing iterators over `LinkedList` objects:

```
private class LRIterator
    implements Iterator<E> {
    // An LRIterator object is a left-to-right iterator over
    // a LinkedList<E> object.

    private Node<E> position;
    // position is a link to the node containing the next
    // element to be visited.

    private LRIterator () {
        position = first;
    }
```

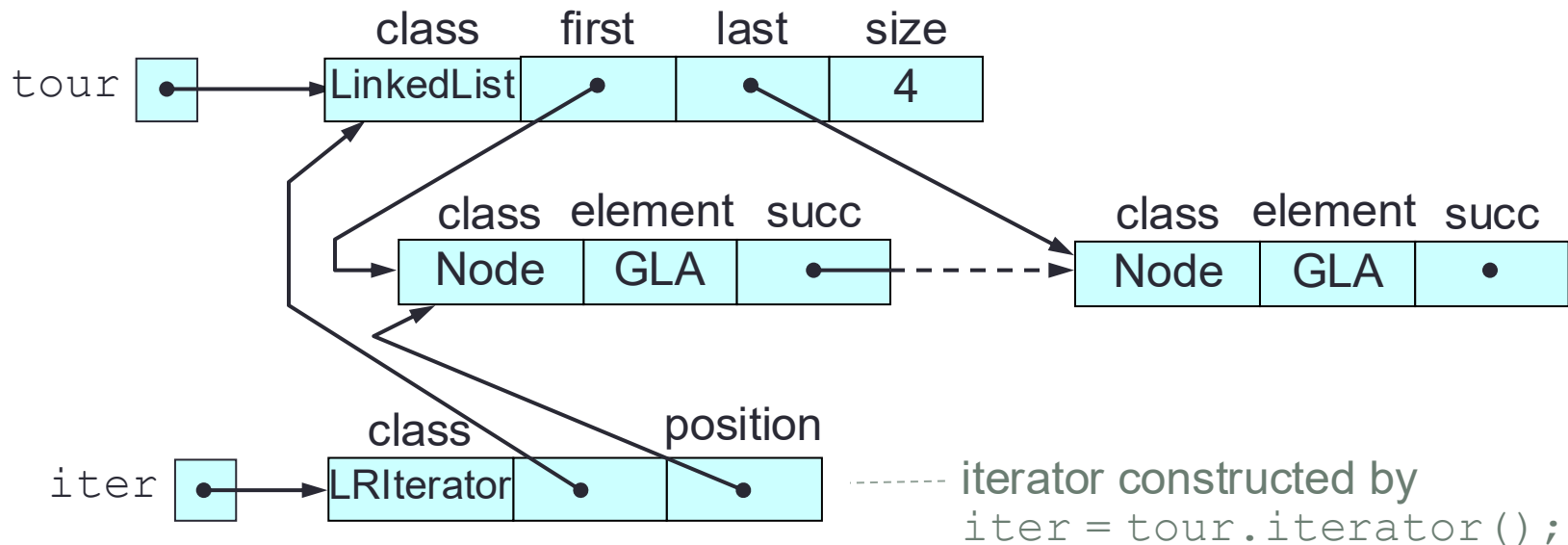
Implementation of lists using SLLs (8)

- Implementing iterators over `LinkedList` objects
(*continued*):

```
public boolean hasNext () {  
    return (position != null);  
}  
  
public E next () {  
    if (position == null) throw ...;  
    E nextElem = position.element;  
    position = position.next;  
    return nextElem;  
}  
...  
}
```

Implementation of lists using SLLs (9)

- Since `LRIterator` is a non-static inner class of `LinkedList`, its methods can access `LinkedList` instance variables:



Summary of list implementations

- Time complexities of main operations:

Operation		Array representation	SLL representation
get		$O(1)$	$O(p)$
set		$O(1)$	$O(p)$
remove		$O(n)$	$O(p)$
add		$O(n)$	$O(p)$
addLast	best	$O(1)$	$O(1)$
	worst	$O(n)$	
equals		$O(n)$	$O(n)$
addAll	best	$O(n')$	$O(1)$
	worst	$O(n+n')$	

where n' = size of second list

Assuming
have
reference
to last node
of first list,
just redirect
to first
node of
second list

Iterating over a list with a Java for-loop

- The following code pattern is extremely common:

```
List<T> list;  
...  
Iterator<T> elems = list.iterator();  
while (elems.hasNext()) {  
    T elem = elems.next();  
    ... elem ...  
}
```

- So Java provides equivalent for-loop notation:

```
List<T> list;  
...  
for (T elem : list) {  
    ... elem ...  
}
```

Read this as “for each element `elem` in `list`, do the following”.

Lists in the Java class library (1)

- The library interface `java.util.List<E>` is similar to the above interface `List<E>`.
- The library class `java.util.ArrayList<E>` implements `java.util.List<E>`, representing each list by an array.
- The library class `java.util.LinkedList<E>` implements `java.util.List<E>`, representing each list by a *doubly-linked-list*. (Why?)

Lists in the Java class library (2)

- Time complexities of the principal list methods:

Method	ArrayList	LinkedList
get	$O(1)$	$O(p)$
set	$O(1)$	$O(p)$
remove	$O(n)$	$O(p)$
add	$O(n)$	$O(p)$
addLast	$O(1)$	$O(1)$
best	$O(1)$	$O(1)$
worst	$O(n)$	$O(1)$
amortized	$O(1)$	$O(1)$

攤銷的

Aside: amortized complexity (1)

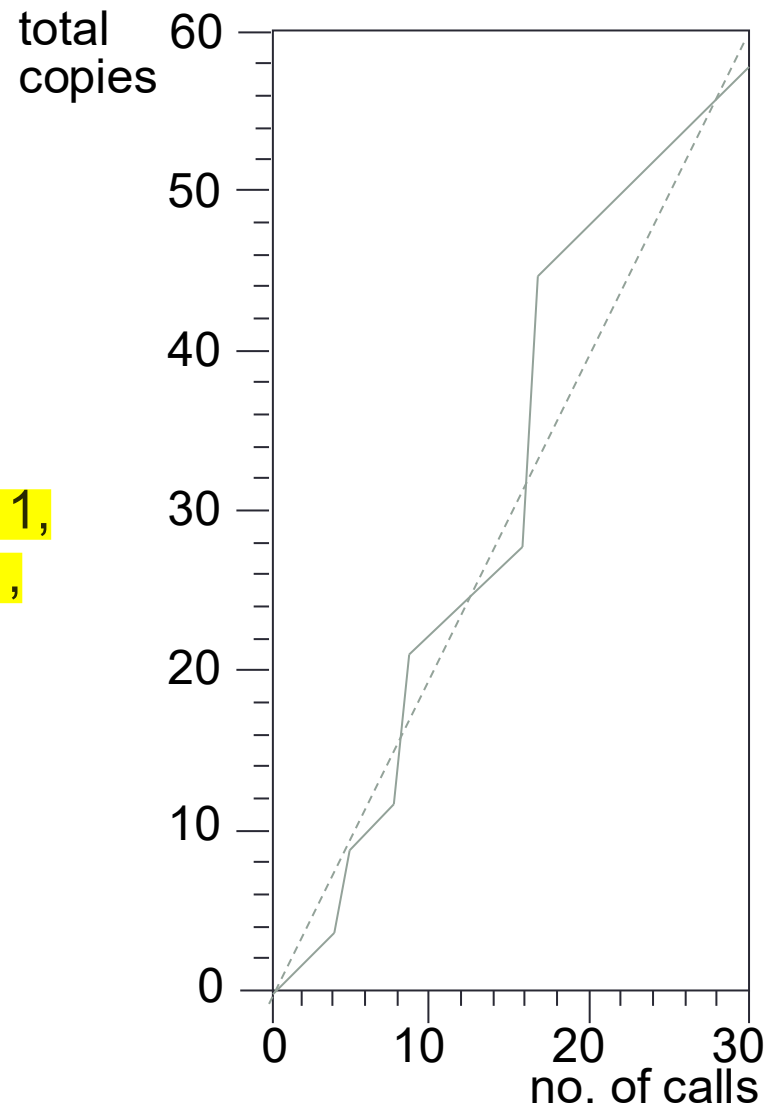
- An operation's **amortized** time complexity reflects its performance averaged over a large number of calls.
- Consider the `addLast` method in `ArrayList` :
 - Normally, only 1 copy is needed.
 - When the array is full, n elements are copied into a new array with doubled length, so in total $n+1$ copies are needed.

Aside: amortized complexity (2)

- Consider 30 consecutive additions to an empty list (with initial capacity 4).
- Number of copies:

1, 1, 1, 1, 5, 1, 1, 1, 9, 1, 1, 1, 1,
 1, 1, 1, 17, 1, 1, 1, 1, 1, 1, 1, 1,
 1, 1, 1, 1, 1 (total 58)

- On average:
 ≈ 2 copies per call.
- Amortized time complexity is
 $O(1)$.



Example: simple text editor again

- See Moodle